Sorbonne Université

Sciences et Ingénierie

MSc Computer Science – MIND

Large Language Models

Comparaison LoRA vs QLoRA

Antoine Théologien

21400184



Encadrant :	Benjamin Piwowarski
Cours :	Large Language Models
Date :	1er janvier 2026

1 Introduction

Les grands modèles de langage (LLMs) sont aujourd'hui des outils puissants pouvant répondre à un grande nombre de tâche axées sur le traitement du langage naturel. La taille croissante de ces modèles rend en revanche le fine-tuning complet coûteux, tant en mémoire qu'en temps. Afin d'outrepasser ce problème, nous nous intéressons à deux méthodes distinctes portant sur l'optimisation de paramètres : **LoRA** et **QLoRA**.

Dans ce rapport, nous comparons ces deux approches en termes de performances, de consommation mémoire et de temps d'entraînement, en utilisant la librairie `PEFT`. Les expériences sont menées sur deux tâches de classification afin d'évaluer la robustesse et la généralisabilité des méthodes étudiées.

2 Méthodologie

Les expériences sont réalisées à partir du modèle `distilbert-base-uncased`, une version compacte de BERT largement utilisée pour les tâches de classification. Le modèle est entraîné et évalué sur un environnement Google Colab équipé d'un GPU NVIDIA T4.
Trois configurations sont comparées :

- **Fine-tuning complet (Baseline)** : l'ensemble des paramètres du modèle est mis à jour.
- **LoRA** : des matrices de faible rang sont injectées dans certaines couches d'attention, tandis que les poids d'origine sont gelés.
- **Configuration inspirée de QLoRA** : une combinaison de LoRA et de techniques de réduction de précision visant à limiter la consommation mémoire, avec une quantification 4 bits.

Les modèles sont évalués sur deux jeux de données standards :

- **IMDB**, pour une tâche de classification binaire de sentiment.
- **AG News**, pour une tâche de classification multi-classes comportant quatre catégories.

3 Métriques d'évaluation

Afin de comparer de manière objective les différentes méthodes de fine-tuning, plusieurs métriques complémentaires ont été utilisées. Elles permettent d'évaluer à la fois les performances prédictives des modèles et leur coût computationnel.

3.1 Consommation mémoire GPU

La consommation mémoire ($VRAM$) correspond à la quantité maximale de mémoire GPU utilisée lors de l'entraînement du modèle. Cette métrique est essentielle pour évaluer l'intérêt pratique des méthodes de fine-tuning efficaces, notamment dans des environnements contraints en ressources.

Elle inclut :

- les poids du modèle,
- les paramètres entraînables,
- les gradients et états de l'optimiseur.

3.2 Temps d'entraînement

Le temps d'entraînement correspond à la durée nécessaire pour effectuer une époque complète sur l'ensemble du jeu de données d'entraînement. Il est mesuré en secondes et permet de comparer l'efficacité computationnelle des différentes approches.

Cette métrique peut être influencée par le nombre de paramètres entraînables, la présence ou non de quantization etc.

À cela s'ajoute la vitesse d'inférence, mesure pouvant être utilisée dans le cas de mise en production, qui n'a été mise en place que pour le dataset IMDB, l'impact n'étant pas très significatif.

3.3 Proportion de paramètres entraînables

Enfin, nous rapportons également la proportion de paramètres effectivement mis à jour durant l'entraînement. Cette métrique met en évidence le gain apporté par les méthodes de fine-tuning efficaces par rapport au fine-tuning complet, et permet de mieux comprendre le compromis entre expressivité du modèle et coût computationnel.

4 Résultats : Classification Binaire (IMDB)

Le Tableau 1 présente les performances obtenues sur le jeu de données IMDB. Les métriques considérées incluent la précision finale, la consommation mémoire GPU et le temps d'entraînement.

Méthode	**Accuracy**	**VRAM (GB)**	**Temps (s)**	**Params entraînés**
Fine-tuning complet	0.912	3.20	45.0	100%
LoRA ($r = 8$)	0.908	1.10	38.5	∼0.6%
Config. inspirée QLoRA ($r = 8$)	0.905	**0.80**	52.0	∼0.6%

TABLE 1 – Comparaison des méthodes sur IMDB

Les résultats montrent que LoRA permet de conserver des performances (évaluées par l'accuracy) proches du fine-tuning complet tout en réduisant énormément le nombre de paramètres entraînables. La configuration inspirée de QLoRA permet une réduction supplémentaire de la mémoire GPU, au prix d'un temps d'entraînement plus élevé, ce qui peut être dû à l'implémentation.

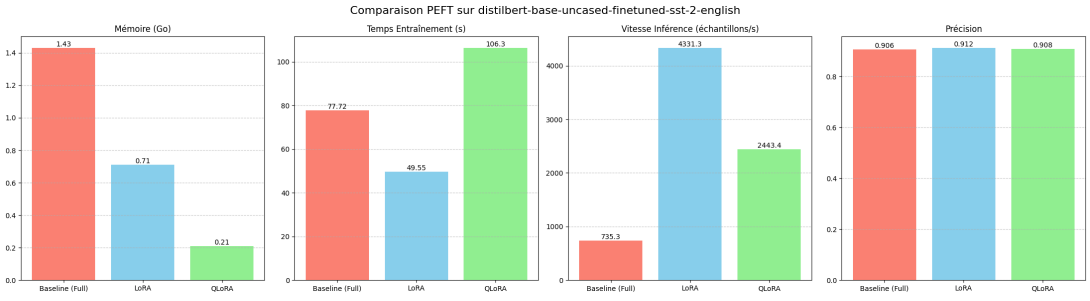


FIGURE 1 – Comparaison des métriques d'entraînement sur IMDB

5 Analyse de Sensibilité

Une analyse de sensibilité est menée afin d'évaluer l'impact du rang $r \in \{4, 8, 16, 32, 64, 128\}$ des matrices LoRA sur nos différentes métriques, entre nos deux méthodes principales, LoRA et QLoRA.

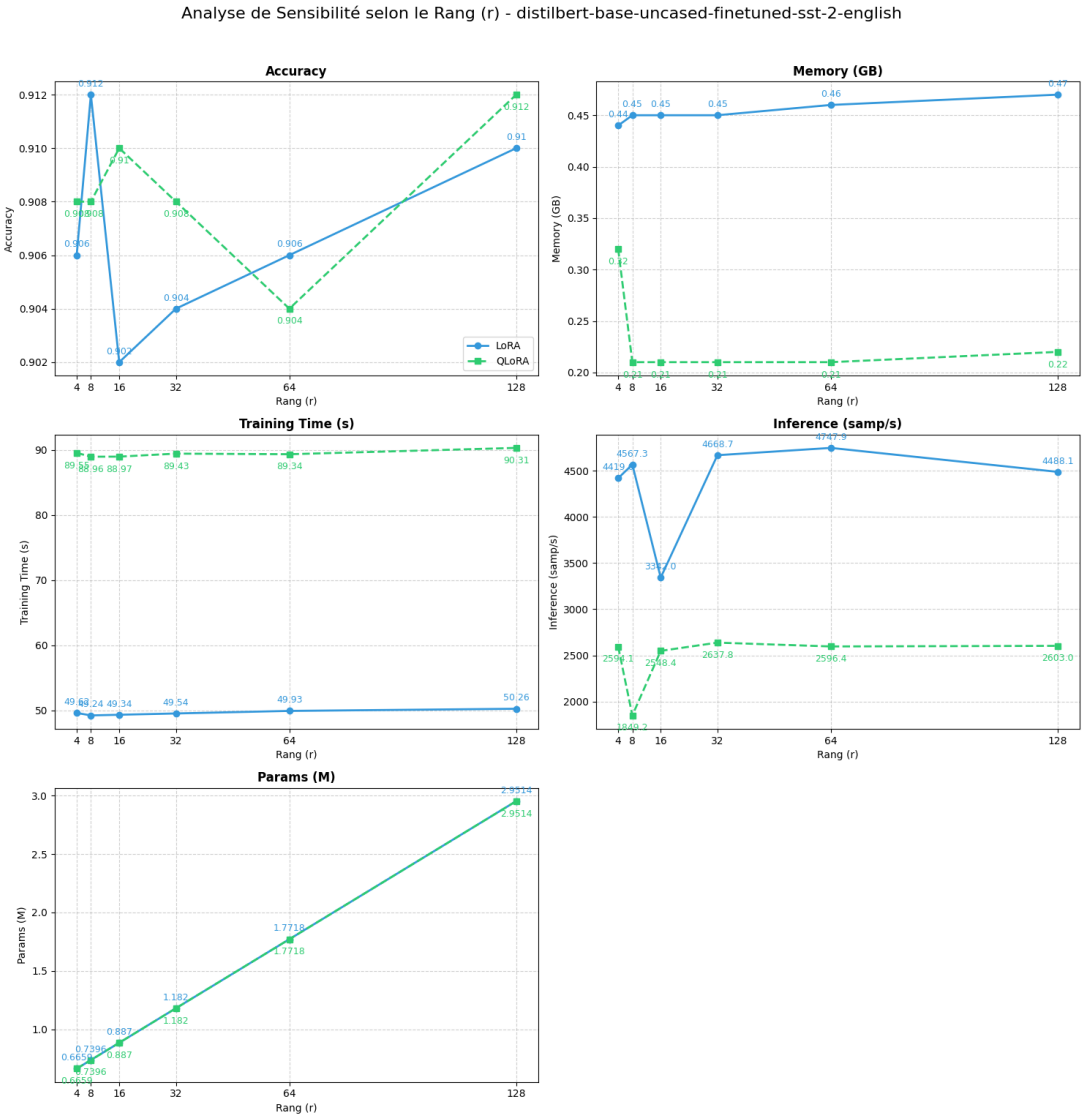


FIGURE 2 – Impact du rang r sur les différentes métriques

On observe ainsi que la valeur d'accuracy n'évolue pas de la même manière pour nos deux méthodes. En effet, pour LoRA, le rang le plus optimal semble être de 8, tandis qu'un rang de 128 semble être beaucoup plus performant pour QLoRA. De plus, pour l'usage mémoire et les temps d'entraînement et d'inférence, l'évolution du rang ne change pas grand chose, montrant qu'il ne s'agit pas d'un hyperparamètre important à optimiser si l'objectif est de gagner en temps ou en ressource.

6 Généralisation : AG News

Les mêmes configurations sont appliquées au jeu de données AG News afin d'évaluer la capacité de généralisation des méthodes étudiées. Malgré le changement de tâche et le passage à une classification multi-classes, les deux approches conservent des performances toutes aussi compétitives. Les proportions des métriques sont également les mêmes, sauf pour le temps d'inférence, pour lequel la vitesse d'inférence semble être cette fois ci meilleure pour le fine-tuning complet.

Pour l'analyse de sensibilité, les résultats sont les mêmes : le rang à son importance uniquement pour l'accuracy, et non pour les autres métriques utilisées.

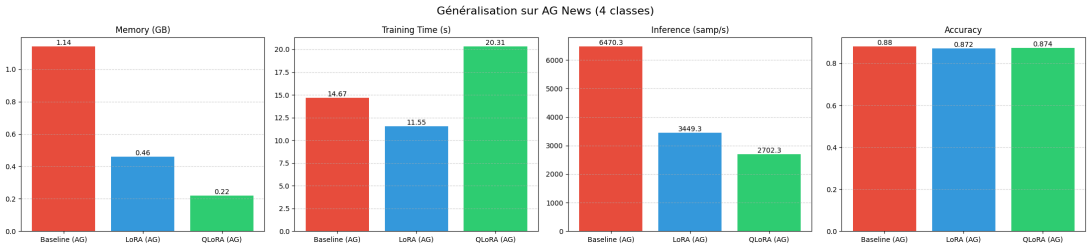


FIGURE 3 – Comparaison des métriques d'entraînement sur AG NEWS

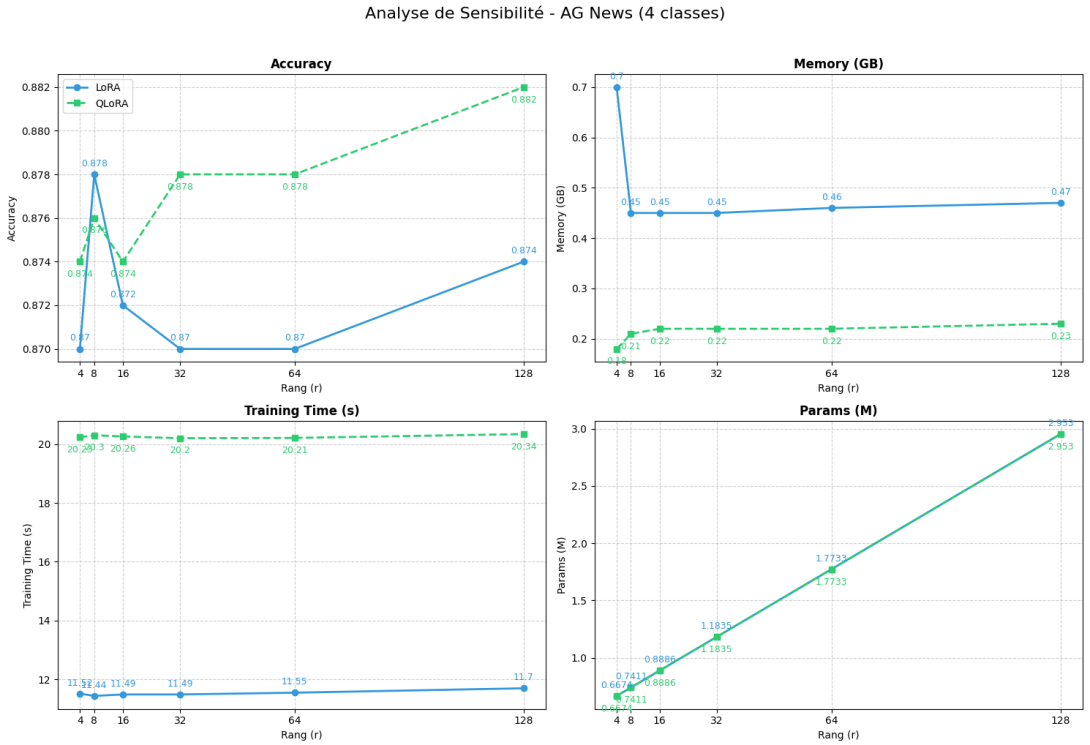


FIGURE 4 – Évolution des métriques d'entraînement sur AG NEWS

7 Conclusion

Ce travail expérimental met en évidence l'intérêt des méthodes de fine-tuning pour l'adaptation de modèles de langage à des tâches de classification. Les deux méthodes permettent de réduire le

nombre de paramètres entraînables en conservant des performances proches.

La configuration QLoRA nous montre également que la quantization peut être fortement utile afin de réduire le coût mémoire, facilitant la mise en place de LLMs sur des infrastructures disposant d'une capacité mémoire plus petite.

Ces résultats confirment que les approches LoRA et QLoRA constituent une bonne solution pour démocratiser l'adaptation de modèles pré-entraînés dans le cadre de LLMs.